Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....5
Требования к реализации.....5
Алгоритм работы программы.....6
1.1 Описание входных данных.....6
1.2 Описание выходных данных.....6
2 МЕТОД РЕШЕНИЯ.....7
3 ОПИСАНИЕ АЛГОРИТМОВ.....8
3.1 Алгоритм метода setHealth класса Player.....8
3.2 Алгоритм метода getHealth класса Player.....8
3.3 Алгоритм метода changeHealth класса Player.....9
3.4 Алгоритм функции main.....9
3.5 Алгоритм функции modifyByPointer.....10
3.6 Алгоритм функции modifyByValue.....10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....12
5 КОД ПРОГРАММЫ.....15
5.1 Файл main.cpp.....15
5.2 Файл Player.cpp.....16
5.3 Файл Player.h.....16
6 ТЕСТИРОВАНИЕ.....17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....18

1 ПОСТАНОВКА ЗАДАЧИ

Разработать программную систему, моделирующую игрового персонажа и демонстрирующую различие между передачей объекта в функцию по указателю и по значению.

Программная система содержит **один класс**. Класс должен включать:

- одно закрытое целочисленное свойство значения здоровья персонажа;
- открытые методы установки и получения значения здоровья;
- открытый метод изменения значения здоровья (значение параметра прибавляется к значению здоровья).

Требования к реализации

- В программе должна быть реализована функция, принимающая **указатель на объект**.
- В программе должна быть реализована функция, принимающая **объект**.
- В обеих функциях должно использоваться одинаковое условие изменения значения здоровья:
    - если значение здоровья меньше 50, здоровье увеличивается на 10;
    - если значение здоровья больше или равно 50, здоровье уменьшается на 5.
- Освобождение динамически выделенной памяти является обязательным.
- Использование передачи по ссылке запрещено.

Алгоритм работы программы

1. Динамически создать объект класса.
2. Ввести начальное значение здоровья.
3. Установить значение здоровья через метод класса.
4. Передать указатель на объект в функцию.
5. Передать объект в функцию.
6. Вывести итоговое значение здоровья.
7. Освободить выделенную память.
8. Завершить работу программы.

1.1 Описание входных данных

Одно целое число — начальное значение здоровья персонажа.

1.2 Описание выходных данных

HealthPoint: "Одно целое число — итоговое значение здоровья персонажа".

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект PlayerPtr класса Player предназначен для указания на динамически созданный объект персонажа;
- функция modifyByPointer для изменения здоровья объекта, переданного по указателю;
- функция modifyByValue для демонстрации передачи объекта по значению. Изменения применяются только к копии;
- функция main для основная функция программы.

Класс Player:

- свойства/поля:
  - поле поле целочисленное значения здоровья персонажа:
    - наименование — health;
    - тип — целочисленный;
    - модификатор доступа — private;
- функционал:
  - метод setHealth — установка значения здоровья персонажа;
  - метод getHealth — получение текущего значения здоровья персонажа;
  - метод changeHealth — изменение значения здоровья персонажа на величину параметра.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода setHealth класса Player

Функционал: установка значения здоровья персонажа.

Параметры: целочисленное число h, новое значение здровья.

Возвращаемое значение: нет.

Алгоритм метода представлен в таблице 1.

Таблица 1 – Алгоритм метода setHealth класса Player

№	Предикат	Действия	№ перехода
1		Присвоить полю health значение параметра h	∅

3.2 Алгоритм метода getHealth класса Player

Функционал: получение текущего значения здоровья персонажа.

Параметры: нет.

Возвращаемое значение: целочисленное значение, текущее значение здровья персонажа.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода getHealth класса Player

№	Предикат	Действия	№ перехода
1		вернуть значение поля health	∅

3.3 Алгоритм метода changeHealth класса Player

Функционал: изменение значения здоровья персонажа на величину параметра.

Параметры: целочисленное value, величина изменения здоровья.

Возвращаемое значение: нет.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода changeHealth класса Player

№	Предикат	Действия	№ перехода
1		Прибавить значение параметра value к полю health	∅

3.4 Алгоритм функции main

Функционал: основная функция программы.

Параметры: нет.

Возвращаемое значение: целочисленное значение, код завершения программы.

Алгоритм функции представлен в таблице 4.

Таблица 4 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		Объявление целочисленной переменной initialHealth	2
2		Ввод значения с клавиатуры в переменную initialHealth	3
3		создать указатель playerPtr на объект класса Player	4
4		Вызов метода setHealth для объекта playerPtr с аргументом initialHealth	5
5		Вызов функции modifyByPointer с аргументом playerPtr (передача объекта по указателю)	6
6		Вызов функции modifyByValue с аргументом *playerPtr(передача	7

№	Предикат	Действия	№ перехода
		объекта по значению, создаётся копия)	
7		Вывод сообщения "HealthPoint: " и текущего здровья, полученного методом platerPtr->getHealth()	8
8		удаление указателя playerPtr на объект класса Player	9
9		Возврат 0	∅

3.5 Алгоритм функции modifyByPointer

Функционал: Изменение здоровья объекта, переданного по указателю. Если текущее здоровье меньше 50, оно увеличивается на 10. Если текущее здоровье больше или равно 50, оно уменьшается на 5.

Параметры: указатель p на объект класса.

Возвращаемое значение: нет.

Алгоритм функции представлен в таблице 5.

Таблица 5 – Алгоритм функции modifyByPointer

№	Предикат	Действия	№ перехода
1	p->getHealth() < 50	Вызвать метод p-> changeHealth(10)	∅
		Вызвать метод p-> changeHealth(-5)	∅

3.6 Алгоритм функции modifyByValue

Функционал: Демонстрация передачи объекта по значению. Изменения применяются только к локальной копии объекта и не влияют на оригинал.

Параметры: объект p класса Player(передаётся по значению).

Возвращаемое значение: нет.

Алгоритм функции представлен в таблице 6.

Таблица 6 – Алгоритм функции modifyByValue

№	Предикат	Действия	№ перехода
1	p.getHealth() < 50	Вызвать метод p.changeHealth(10)	∅
		Вызвать метод p.changeHealth(-5)	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-3.

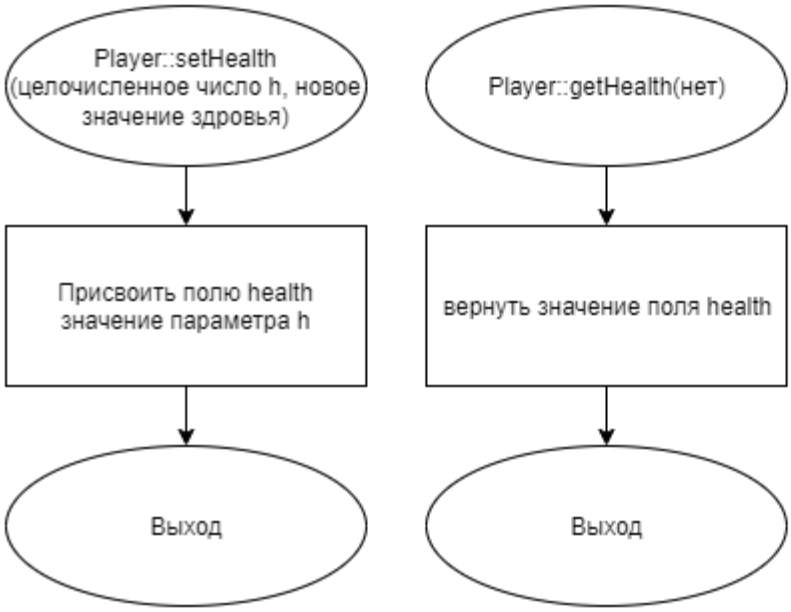

Рисунок 1 – Блок-схема алгоритма

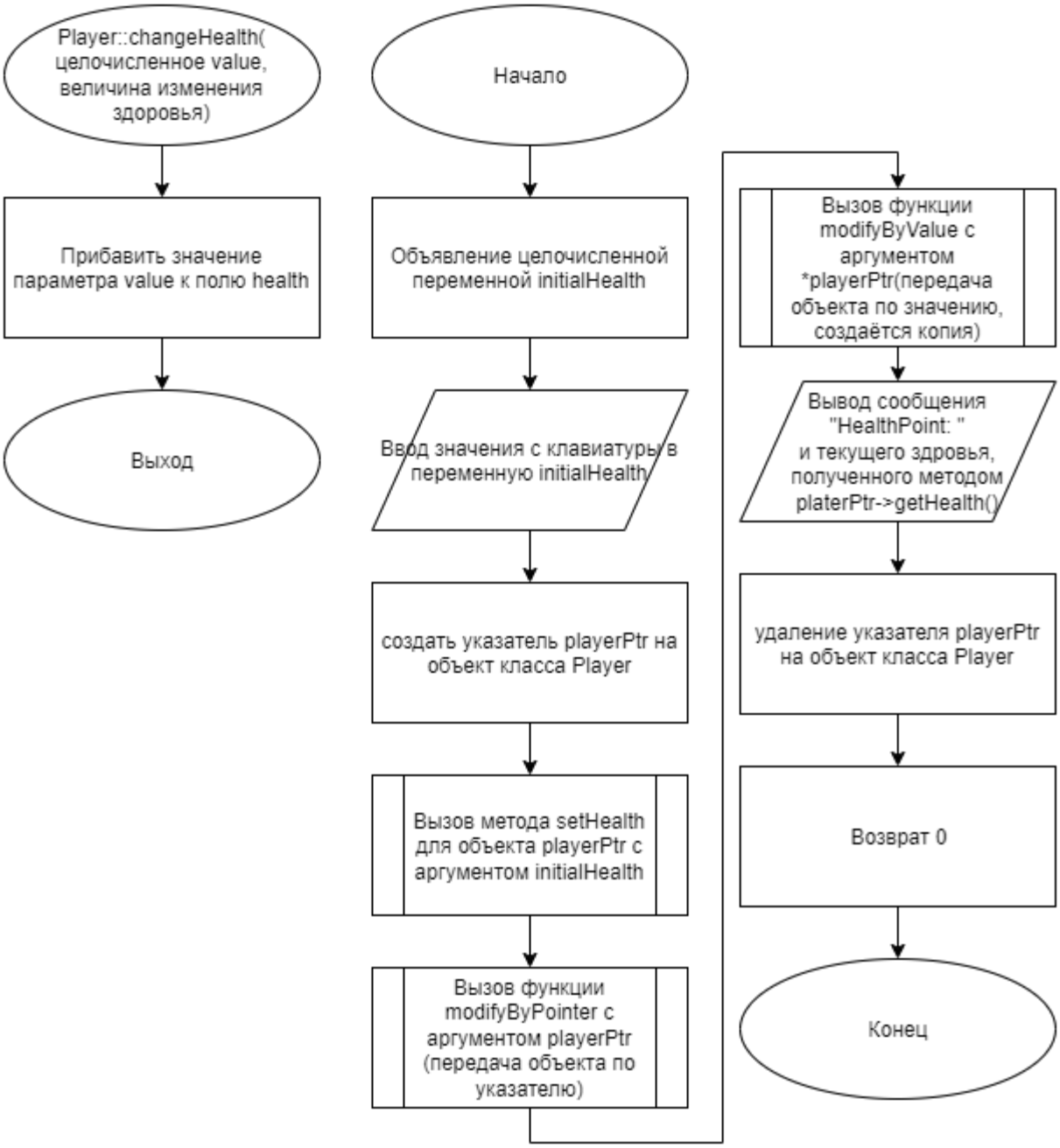

Рисунок 2 – Блок-схема алгоритма

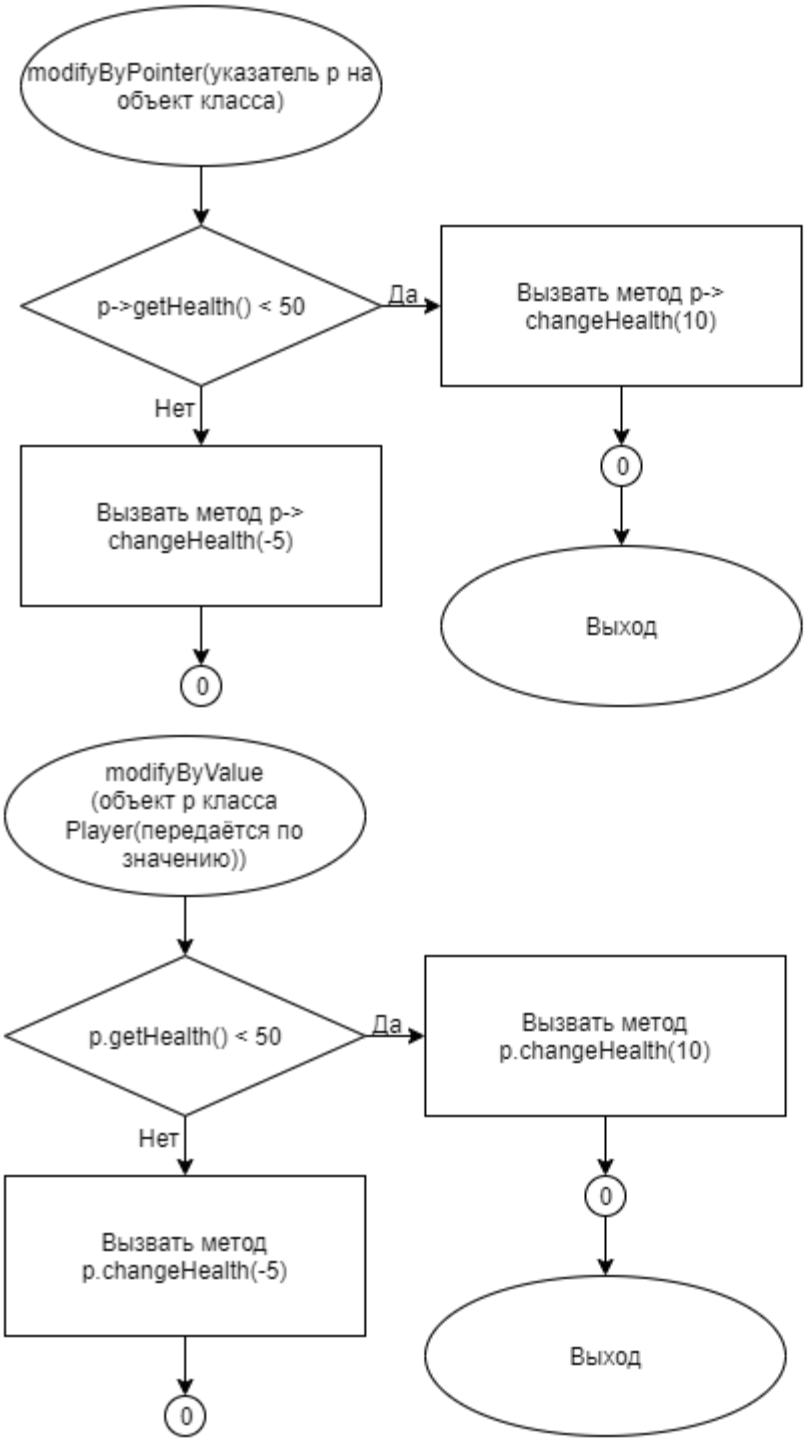

Рисунок 3 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл main.cpp

Листинг 1 – main.cpp

```
#include <iostream>
#include "Player.h"
using namespace std;

void modifyByPointer(Player* p){
   if (p -> getHealth() < 50){
      p -> changeHealth(10);
   }
   else{
      p -> changeHealth(-5);
   }
}

void modifyByValue(Player p){
   if (p.getHealth() < 50){
      p.changeHealth(10);
   }
   else{
      p.changeHealth(-5);
   }
}

int main()
{
   int initialHealth;
   cin >> initialHealth;

   Player* playerPtr = new Player();

   playerPtr -> setHealth(initialHealth);

   modifyByPointer(playerPtr);

   modifyByValue(*playerPtr);

   cout << "HealthPoint: " << playerPtr->getHealth() << endl;

   delete playerPtr;
```

```
        return(0);
    }
```

5.2 Файл Player.cpp

Листинг 2 – Player.cpp

```
    #include "Player.h"

    void Player::setHealth(int h){
        health = h;
    }

    int Player::getHealth() const {
        return health;
    }

    void Player::changeHealth(int value){
        health += value;
    }
```

5.3 Файл Player.h

Листинг 3 – Player.h

```
    #ifndef __PLAYER__H
    #define __PLAYER__H

    class Player {
    private:
        int health;

    public:
        void setHealth(int h);
        int getHealth() const;
        void changeHealth(int value);

    };

    #endif
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 7.

Таблица 7 – Результат тестирования программы

Входные данные	**Ожидаемые выходные данные**	**Фактические выходные данные**
`50`	`HealthPoint: 45`	`HealthPoint: 45`
`45`	`HealthPoint: 55`	`HealthPoint: 55`

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.

2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avrora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).

3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avrora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).

4. Шилдт Г. C++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.

5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».

6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).